

ULTIMATE PRACTICE CHEAT SEET OF SQL JOINS

➤ Dataset Creation

• Students Table

```
1 CREATE TABLE Students (  
2     student_id INT PRIMARY KEY,  
3     student_name VARCHAR(50),  
4     department_id INT  
5 );  
6  
7 • INSERT INTO Students (student_id, student_name, department_id)  
8 VALUES  
9     (1, 'Shantanu', 101),  
10    (2, 'Priya', 102),  
11    (3, 'Rahul', 101),  
12    (4, 'Riya', 103),  
13    (5, 'Arjun', NULL);  
14
```

Result Grid Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

	student_id	student_name	department_id
▶	1	Shantanu	101
	2	Priya	102
	3	Rahul	101
	4	Riya	103
	5	Arjun	NULL
*	NULL	NULL	NULL

students 1 x

• Departments Table

```
1 CREATE TABLE Departments (  
2     department_id INT PRIMARY KEY,  
3     department_name VARCHAR(50)  
4 );  
5  
6 • INSERT INTO Departments (department_id, department_name)  
7 VALUES  
8     (101, 'Computer Science'),  
9     (102, 'Marketing'),  
10    (103, 'Finance'),  
11    (104, 'HR');  
12
```

Result Grid Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

	department_id	department_name
▶	101	Computer Science
	102	Marketing
	103	Finance
	104	HR
*	NULL	NULL

departments 1 x

- Results Tables

```
1 CREATE TABLE Results (  
2     result_id INT PRIMARY KEY,  
3     student_id INT,  
4     marks INT  
5 );  
6  
7 • INSERT INTO Results (result_id, student_id, marks)  
8 VALUES  
9 (1, 1, 85),  
10 (2, 2, 75),  
11 (3, 3, 92),  
12 (4, 5, 60);  
13
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

	result_id	student_id	marks
▶	1	1	85
	2	2	75
	3	3	92
	4	5	60
*	NULL	NULL	NULL

results 1 x

- Managers Table (for Self Join)

```
1 CREATE TABLE Managers (  
2     emp_id INT PRIMARY KEY,  
3     emp_name VARCHAR(50),  
4     manager_id INT  
5 );  
6  
7 • INSERT INTO Managers (emp_id, emp_name, manager_id)  
8 VALUES  
9 (1, 'Raj', NULL),  
10 (2, 'Rohan', 1),  
11 (3, 'Shantanu', 2),  
12 (4, 'Priya', 2);  
13
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

	emp_id	emp_name	manager_id
▶	1	Raj	NULL
	2	Rohan	1
	3	Shantanu	2
	4	Priya	2
*	NULL	NULL	NULL

managers 1 x

➤ INNER JOIN

- Shows only matching rows from both tables based on a common column.

Q1) Show each student's name along with their department name.

- Solution and output

```
1 • select students.student_name, departments.department_name
2   from students inner join departments
3   on students.department_id = departments.department_id;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
student_name	department_name		
Shantanu	Computer Science		
Priya	Marketing		
Rahul	Computer Science		
Riya	Finance		

Q2) Display the student name and marks (use Students and Results tables).

- Solution and output

```
1 • select students.student_name, results.marks
2   from students inner join results
3   on students.student_id = results.student_id;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
student_name	marks		
Shantanu	85		
Priya	75		
Rahul	92		
Arjun	60		

Q3) Show department name and the names of students who belong to that department.

- Solution and output

```
1 • select departments.department_name, students.student_name
2   from departments inner join students
3   on departments.department_id = students.department_id;
4
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
department_name	student_name		
Computer Science	Shantanu		
Marketing	Priya		
Computer Science	Rahul		
Finance	Riya		

➤ LEFT JOIN

- Shows all rows from the left table, and only the matching rows from the right table. Unmatched rows from the right side become NULL.

Q1) Display all students and their department names — include those who don't belong to any department.

- Solution and output

```
1 • select * from students
2 left join departments
3 on students.department_id = departments.department_id;
4
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	student_id	student_name	department_id	department_id	department_name
▶	1	Shantanu	101	101	Computer Science
	2	Priya	102	102	Marketing
	3	Rahul	101	101	Computer Science
	4	Riya	103	103	Finance
	5	Arjun	NULL	NULL	NULL

Q2) List all students and their marks, even if some students don't have marks yet.

- Solution and output

```
1 • select students.student_id , students.student_name, results.marks
2 from students left join results
3 on students.student_id = results.student_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	student_id	student_name	marks
▶	1	Shantanu	85
	2	Priya	75
	3	Rahul	92
	4	Riya	NULL
	5	Arjun	60

➤ RIGHT JOIN

- Shows all rows from the right table, and only matching rows from the left table. Unmatched rows from the left side become NULL.

Q1) Show all departments and their student names (include departments with no students).

- Solution and output

```
1 • select departments.department_name, students.student_name
2   from departments right join students
3   on departments.department_id = students.department_id;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
department_name	student_name		
Computer Science	Shantanu		
Marketing	Priya		
Computer Science	Rahul		
Finance	Riya		
NULL	Arjun		

Q2) Display all students' marks from the Results table — even if some marks don't belong to a valid student.

- Solution and output

```
1 • select students.student_name, results.marks
2   from results right join students
3   on results.student_id = students.student_id;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
student_name	marks		
Shantanu	85		
Priya	75		
Rahul	92		
Riya	NULL		
Arjun	60		

➤ FULL OUTER JOIN

- Shows all rows from both tables, matching where possible.
If no match, fills with NULL on either side.

(Note: If you use MySQL, use LEFT JOIN + RIGHT JOIN + UNION to simulate FULL OUTER JOIN)

Q1) Show all students and departments — include even those without any match.

- Solution and output

```
1 • select s.student_name, d.department_name
2   from students s
3  left join departments d
4    on s.department_id = d.department_id
5  union
6  select s.student_name, d.department_name
7   from students s
8  right join departments d
9    on s.department_id = d.department_id;
10
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	student_name	department_name
▶	Shantanu	Computer Science
	Priya	Marketing
	Rahul	Computer Science
	Riya	Finance
	Arjun	NULL
	NULL	HR

Q2) Combine student and result data to show all students and all marks, whether matched or not.

- Solution and output

```
2 • select s.student_name, r.marks
3   from students s
4  left join results r
5    on s.student_id = r.student_id
6  union
7  select s.student_name, r.marks
8   from students s
9  right join results r
10    on s.student_id = r.student_id;
11
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	student_name	marks
▶	Shantanu	85
	Priya	75
	Rahul	92
	Riya	NULL
	Arjun	60

➤ CROSS JOIN

- Returns all possible combinations of rows from both tables.
(Also called a Cartesian Product)

Q1) Display all possible combinations of students and departments.

- Solution and output

```
1 • select students.student_name, departments.department_name
2   from students
3  cross join departments;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	student_name	department_name
▶	Shantanu	HR
	Shantanu	Finance
	Shantanu	Marketing
	Shantanu	Computer Science
	Priya	HR
	Priya	Finance
	Priya	Marketing
	Priya	Computer Science
	Rahul	HR
	Rahul	Finance
	Rahul	Marketing
	Rahul	Computer Science
	Riya	HR
	Riya	Finance
	Riya	Marketing
	Riya	Computer Science
	Arjun	HR
	Arjun	Finance
	Arjun	Marketing
	Ariun	Computer Science

Q2) Generate a list of all possible student-department pairs (useful to simulate assigning students to every department).

- Solution and output

```
1 • select CONCAT(students.student_name, ' - ', departments.department_name)
2   as student_department_pair
3   from students
4  cross join departments;
5
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	student_department_pair
▶	Shantanu - HR
	Shantanu - Finance
	Shantanu - Marketing
	Shantanu - Computer Science
	Priya - HR
	Priya - Finance
	Priya - Marketing
	Priya - Computer Science
	Rahul - HR
	Rahul - Finance
	Rahul - Marketing
	Rahul - Computer Science
	Riya - HR
	Riya - Finance
	Riya - Marketing
	Riya - Computer Science
	Arjun - HR
	Arjun - Finance
	Arjun - Marketing
	Ariun - Computer Science

➤ SELF JOIN

- Joins a table with itself to compare related rows within the same table.

Q1) Using the Managers table, display each employee with their manager's name.

- Solution and output

```
1 • select e.emp_name as Employee, m.emp_name as Manager
2   from managers e
3  left join managers m
4    on e.manager_id = m.emp_id;
```

	Employee	Manager
▶	Raj	NULL
	Rohan	Raj
	Shantanu	Rohan
	Priya	Rohan

Q2) Show all employees who are managers of someone.

- Solution and output

```
1 • select distinct m.emp_name as Manager
2   from managers e
3  inner join managers m
4    on e.manager_id = m.emp_id;
```

	Manager
▶	Raj
	Rohan

Q3) Find the employee-manager hierarchy (who reports to whom).

- Solution and output

```
1 • select e.emp_name as Employee, m.emp_name as Manager, mm.emp_name as Senior_Manager
2   from managers e
3  left join managers m on e.manager_id = m.emp_id
4  left join managers mm on m.manager_id = mm.emp_id;
5
```

	Employee	Manager	Senior_Manager
▶	Raj	NULL	NULL
	Rohan	Raj	NULL
	Shantanu	Rohan	Raj
	Priya	Rohan	Raj